ATIVIDADE INTEGRADORA — JAVA E POO

Tema: Sistema Acadêmico Simplificado | Duração: 3 horas | Modalidade: trios | Ambiente: VS Code e JDK 21

Objetivo: revisar, explicar e aplicar os conteúdos das Aulas 01 a 08. A atividade é dividida em duas partes de 90 minutos: questões teóricas e construção guiada de um sistema Java.

Parte I — Questões teóricas (90 minutos)

Bloco A — Fundamentos de POO

1. Explique a diferença entre classe e objeto. Apresente um exemplo de cada um em um sistema acadêmico.

2. O que são atributos e métodos? Diferencie estado e comportamento de um objeto.

3. Qual é a função de new? Diferencie criar um objeto e declarar uma referência.

4. Explique encapsulamento. Por que evitar atributos públicos sem necessidade?

5. Qual é a finalidade do construtor? Explique o uso de this.

Bloco B — Relacionamentos e herança

6. Diferencie associação, agregação e composição; exemplifique.

7. O que é ArrayList e quando preferi-lo a um vetor de tamanho fixo?

8. Diferencie superclasse e subclasse. Explique extends e super.

9. Diferencie sobrecarga e sobrescrita de métodos; dê exemplos.

10. O que é polimorfismo? Como uma referência de superclasse aponta para objetos de subclasses?

Bloco C — Interfaces e projeto

11. O que é interface? Diferencie implements e extends nos exemplos estudados.

12. Compare herança e composição. Quando a composição é preferível?

13. O que é baixo acoplamento? Como a injeção por construtor ajuda?

14. Explique SRP, OCP e DIP com exemplos simples.

Bloco D — Padrões de projeto

15. O que é um padrão de projeto? Por que evitar aplicá-lo indiscriminadamente?

16. Explique Singleton e a finalidade do construtor privado e de getInstancia().

17. Explique Factory Method e o papel dos criadores abstratos e concretos.

18. Como Strategy organiza descontos comuns, estudantis e promocionais?

19. Como Observer comunica um pedido confirmado a estoque, financeiro e auditoria?

20. Diferencie Adapter e Facade; dê um caso de uso de cada um.

Parte II — Construção prática guiada (90 minutos)

Projeto: Sistema Acadêmico Simplificado. O sistema cadastra aluno e curso, aplica desconto, registra pagamento em um serviço legado, notifica observadores e emite comprovante. Os exemplos abaixo usam classes sem package para simplificar a execução.

Cronograma

• 0–15 min: modelagem de Aluno e Curso; Singleton de configuração.

• 15–30 min: Strategy para políticas de desconto.

• 30–45 min: Observer para notificações.

• 45–60 min: Adapter para o sistema de pagamentos.

• 60–75 min: Factory Method e Facade.

• 75–90 min: compilação, testes e análise.

Etapa 1 — Classes, objetos e encapsulamento

Crie Curso.java. O construtor valida os dados; os atributos private final preservam a referência/valor após a inicialização. Os getters oferecem leitura controlada.

```
public class Curso {
    private final String titulo;
    private final double preco;

    public Curso(String titulo, double preco) {
        if (titulo == null || titulo.isBlank() || preco <= 0) {
            throw new IllegalArgumentException("Dados inválidos");
        }
        this.titulo = titulo;
        this.preco = preco;
    }

    public String getTitulo() { return titulo; }
    public double getPreco() { return preco; }
}
```

Crie Aluno.java com nome validado e uma coleção ArrayList<Curso>. Implemente adicionarCurso(Curso), getNome() e getQuantidadeCursos(). A lista representa a associação entre aluno e cursos; não há exigência de destruir Curso quando Aluno deixa de existir.

Etapa 2 — Strategy: políticas de desconto

Crie a interface Desconto.java e duas implementações. A interface estabelece um contrato; o cliente utiliza aplicar sem depender da classe concreta. Em R$ 200,00, a política estudantil resulta em R$ 180,00.

```
public interface Desconto {
    double aplicar(double valor);
}

class SemDesconto implements Desconto {
    public double aplicar(double valor) { return valor; }
}

class DescontoEstudante implements Desconto {
    public double aplicar(double valor) { return valor * 0.90; }
}
```

Etapa 3 — Observer: comunicação de eventos

Crie ObservadorMatricula, AvisoEmail e Auditoria. O serviço manterá uma List<ObservadorMatricula>, permitirá inscrição e chamará atualizar em cada observador depois da matrícula. As mensagens são simuladas no terminal.

```
public interface ObservadorMatricula {
    void atualizar(String mensagem);
}

class AvisoEmail implements ObservadorMatricula {
    public void atualizar(String mensagem) {
        System.out.println("E-mail: " + mensagem);
    }
}

class Auditoria implements ObservadorMatricula {
    public void atualizar(String mensagem) {
        System.out.println("Auditoria: " + mensagem);
    }
}
```

Etapa 4 — Adapter: integração de pagamento

O sistema legado aceita centavos inteiros; a aplicação utiliza reais. O adaptador implementa o contrato esperado e converte a unidade. Para sistemas financeiros reais, substitua double por uma representação monetária adequada, como BigDecimal.

```
class SistemaLegado {
    public void registrarEmCentavos(int centavos) {
        System.out.println("Legado registrou: " + centavos + " centavos");
    }
}

interface RegistradorPagamento {
    void registrar(double reais);
}

class AdaptadorPagamento implements RegistradorPagamento {
    private final SistemaLegado legado;

    public AdaptadorPagamento(SistemaLegado legado) {
        this.legado = legado;
    }

    public void registrar(double reais) {
        int centavos = (int) Math.round(reais * 100);
        legado.registrarEmCentavos(centavos);
    }
}
```

Etapa 5 — Factory Method: comprovantes

Crie a interface Comprovante e duas implementações, ComprovanteTexto e ComprovanteResumido. O criador abstrato define o fluxo emitir e delega a criação para subclasses concretas.

```
interface Comprovante {
    String gerar(Aluno aluno, Curso curso, double valor);
}

abstract class CriadorComprovante {
    public abstract Comprovante criar();

    public String emitir(Aluno aluno, Curso curso, double valor) {
```

```
        return criar().gerar(aluno, curso, valor);
    }
}
```

Implemente CriadorTexto e CriadorResumido, sobrescrevendo criar() para retornar a implementação apropriada. Explique como @Override, herança e polimorfismo tornam possível variar o formato.

Etapa 6 — Facade: simplificar o caso de uso

A fachada expõe uma operação de alto nível. Ela coordena o serviço de matrícula e o criador de comprovantes, mantendo Main simples.

```
class FachadaAcademica {
    private final ServicoMatricula servico;
    private final CriadorComprovante criador;

    public FachadaAcademica(ServicoMatricula servico,
                            CriadorComprovante criador) {
        this.servico = servico;
        this.criador = criador;
    }

    public String matricularEEmitir(Aluno aluno, Curso curso) {
        double total = servico.matricular(aluno, curso);
        return criador.emitir(aluno, curso, total);
    }
}
```

Etapa 7 — Implementação de referência completa

O código a seguir integra todas as etapas, incluindo Singleton, ServicoMatricula e Main. Salve-o como src/Main.java em um projeto separado de referência. Para desenvolver as etapas individualmente, separe as classes em arquivos próprios e evite duplicar declarações.

```
import java.util.ArrayList;
import java.util.List;

interface Desconto { double aplicar(double valor); }
class SemDesconto implements Desconto { public double aplicar(double valor) { return valor; } }
class DescontoEstudante implements Desconto { public double aplicar(double valor) { return
valor * 0.90; } }

final class Configuracao {
    private static final Configuracao INSTANCIA = new Configuracao();
    private final String nome = "Sistema Acadêmico";
    private Configuracao() {}
    public static Configuracao getInstancia() { return INSTANCIA; }
    public String getNome() { return nome; }
}

class Aluno {
    private final String nome;
    private final List<Curso> cursos = new ArrayList<>();
    public Aluno(String nome) {
        if (nome == null || nome.isBlank()) throw new IllegalArgumentException("Nome
inválido");
        this.nome = nome;
    }
    public String getNome() { return nome; }
    public void adicionarCurso(Curso curso) { if (curso == null) throw new
IllegalArgumentException("Curso inválido"); cursos.add(curso); }
    public int getQuantidadeCursos() { return cursos.size(); }
```

```
}

class Curso {
    private final String titulo;
    private final double preco;
    public Curso(String titulo, double preco) {
        if (titulo == null || titulo.isBlank() || preco <= 0) throw new
IllegalArgumentException("Dados inválidos");
        this.titulo = titulo; this.preco = preco;
    }
    public String getTitulo() { return titulo; }
    public double getPreco() { return preco; }
}

interface ObservadorMatricula { void atualizar(String mensagem); }
class AvisoEmail implements ObservadorMatricula {
    public void atualizar(String mensagem) { System.out.println("E-mail: " + mensagem); }
}
class Auditoria implements ObservadorMatricula {
    public void atualizar(String mensagem) { System.out.println("Auditoria: " + mensagem); }
}

interface Comprovante { String gerar(Aluno aluno, Curso curso, double valor); }
class ComprovanteTexto implements Comprovante {
    public String gerar(Aluno aluno, Curso curso, double valor) {
        return "Matrícula: " + aluno.getNome() + " / " + curso.getTitulo() + " / R$ " + valor;
    }
}
class ComprovanteResumido implements Comprovante {
    public String gerar(Aluno aluno, Curso curso, double valor) {
        return aluno.getNome() + " - R$ " + valor;
    }
}
abstract class CriadorComprovante {
    public abstract Comprovante criar();
    public String emitir(Aluno aluno, Curso curso, double valor) {
        return criar().gerar(aluno, curso, valor);
    }
}
class CriadorTexto extends CriadorComprovante {
    public Comprovante criar() { return new ComprovanteTexto(); }
}
class CriadorResumido extends CriadorComprovante {
    public Comprovante criar() { return new ComprovanteResumido(); }
}

class SistemaLegado {
    public void registrarEmCentavos(int centavos) { System.out.println("Legado registrou: " +
centavos + " centavos"); }
}
interface RegistradorPagamento { void registrar(double reais); }
class AdaptadorPagamento implements RegistradorPagamento {
    private final SistemaLegado legado;
    public AdaptadorPagamento(SistemaLegado legado) { this.legado = legado; }
    public void registrar(double reais) { legado.registrarEmCentavos((int) Math.round(reais *
100)); }
}

class ServicoMatricula {
    private final Desconto desconto;
    private final RegistradorPagamento pagamento;
    private final List<ObservadorMatricula> observadores = new ArrayList<>();
    public ServicoMatricula(Desconto desconto, RegistradorPagamento pagamento) {
        this.desconto = desconto; this.pagamento = pagamento;
```

```
    }
    public void inscrever(ObservadorMatricula observador) { observadores.add(observador); }
    public double matricular(Aluno aluno, Curso curso) {
        double total = desconto.aplicar(curso.getPreco());
        pagamento.registrar(total);
        aluno.adicionarCurso(curso);
        for (ObservadorMatricula o : observadores) o.atualizar(aluno.getNome() + " matriculado
em " + curso.getTitulo());
        return total;
    }
}
class FachadaAcademica {
    private final ServicoMatricula servico;
    private final CriadorComprovante criador;
    public FachadaAcademica(ServicoMatricula servico, CriadorComprovante criador) {
        this.servico = servico; this.criador = criador;
    }
    public String matricularEEmitir(Aluno aluno, Curso curso) {
        double total = servico.matricular(aluno, curso);
        return criador.emitir(aluno, curso, total);
    }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(Configuracao.getInstancia().getNome());
        Aluno aluno = new Aluno("Ana");
        Curso curso = new Curso("Java", 200.0);
        ServicoMatricula servico = new ServicoMatricula(new DescontoEstudante(), new
AdaptadorPagamento(new SistemaLegado()));
        servico.inscrever(new AvisoEmail());
        servico.inscrever(new Auditoria());
        FachadaAcademica fachada = new FachadaAcademica(servico, new CriadorTexto());
        System.out.println(fachada.matricularEEmitir(aluno, curso));
        System.out.println("Cursos do aluno: " + aluno.getQuantidadeCursos());
    }
}
```

Como compilar e executar

```
javac -encoding UTF-8 -d out src/Main.java
java -cp out Main
```

Compare a saída com os eventos esperados: registro do pagamento em centavos, notificação por e-mail e auditoria, comprovante e quantidade de cursos do aluno.

Etapa 8 — Perguntas de análise da implementação

1. Qual classe implementa Singleton e quais instruções controlam a instância?

2. Onde aparecem encapsulamento e validação?

3. Qual é a relação entre Aluno e Curso?

4. Onde foi utilizada uma coleção e com qual objetivo?

5. Qual é o papel da interface Desconto?

6. Como acrescentar uma política de 20% sem alterar o serviço?

7. Quem produz o evento e quem recebe as notificações?

8. Por que o adaptador converte reais em centavos?

9. Qual método representa Factory Method?

10. O que FachadaAcademica simplifica para Main?

Critérios de avaliação e entrega

• Questões teóricas: 5,0 pontos.

• Implementação e funcionamento: 3,0 pontos.

• Aplicação correta dos padrões: 1,0 ponto.

• Testes e análise: 1,0 ponto.

• Total: 10,0 pontos.

Entrega: respostas teóricas em PDF; arquivos-fonte .java; captura de tela da execução; respostas às perguntas de análise. O código de referência é material de apoio para estudo e conferência.